

DSA

LAB MANUAL

Submitted to	
Name	Dr. Rajkumar Batchu
Department	CSE
Designation	Assistant Professor
Submitted by	
Name	Edara Ujval Sriteja
Roll No	av.sc.u4cse25107
Year/Sem/Section	2nd year / Sem-3 / CSE-B

INDEX

LAB. NO	QUESTIONS	FROM	TO
1.	**a)** A rocket launch system displays a countdown before launch. Instead using loop implement the countdown from n to 1, followed by "Launch!". **b)** Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P^n (power) where, P = principle growth factor, n = number of years. **c)** An HR department stores employee ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list. **d)** In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number. **e)** Write a recursive python program to print the first n terms of fibonacci series.		
2.	a) Program to implement LINEAR SEARCH b) Program to implement BINARY SEARCH		
3.	a) Write a Program for BUBBLE SORT b) Write a program for INSERTION SORT c)Write a program for SELECTION SORT;		
4.	**4A. Write a Python program to implement Merge Sort, where the elements are taken from the user.** **4B. Write a Python program to implement Quick Sort, where the elements are taken from the user.**		

5.	**5A. Write a program to implement Singly Linked List with the following operations:** **5B. Write a program to implement Doubly Linked List with the following operations:**		
6.	**6A. Write a Python program to implement stack operations such as push, pop, peek, and display using an array and a linked list.** **6B. To implement stack operations such as push, pop, peek, and display using a linked list.**		
7.	7a. Queue implementation using Linked List. 7b. Queue Using Linked List 7c. program for performing the basic operations using circular queue		
8.	Write a Python program to implement a binary tree with its operations using an array and a linked list.	50	57
9.	Write a Python program to implement a Binary Search Tree with its operations and traversals.	58	61

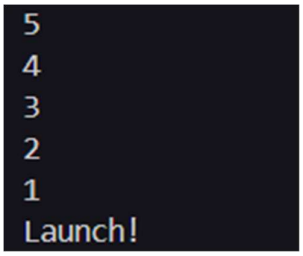
LAB-1

a) A rocket launch system displays a countdown before launch. Instead using loop implement the countdown from n to 1, followed by "Launch!".

CODE:

```
1. def countdown(n):
2.     if n <= 0:
3.         print("Launch!")
4.     else:
5.         print(n)
6.         countdown(n - 1)
7.
8. countdown(5)
```

OUTPUT:



EXPLANATION:

Base Case: The recursion ends when n becomes 0 or a negative value. At that point, the program prints "Launch!".

Recursive Step: For a positive n, the function prints the current value and calls itself again with n reduced by 1. This continues until the base case is reached.

ERROR TABLE:

SNO	ERROR	SOLUTION

No execution errors were encountered.

b) Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P^n (power) where, P = principle growth factor, n = number of years.

CODE:

```
1. def calculate_power(P, n):
2.     if n == 0:
3.         return 1
4.     else:
5.         return P * calculate_power(P, n - 1)
6.

7. print(calculate_power(20, 5))
```

OUTPUT:

3200000

EXPLANATION:

Base Case: When n is 0, the function returns 1 because every number raised to the power 0 is 1.

Recursive Step: For each positive value of n, P is multiplied by the result obtained from calculate_power(P, n - 1). Thus, the multiplication is repeated until n becomes 0.

ERROR TABLE:

SNO	ERROR	SOLUTION
No execution errors were encountered.		

c) An HR department stores employee ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

CODE:

```
1. def search_employee(emp_list, emp_id, index=0):
2.     if index == len(emp_list):
3.         return False
4.     if emp_list[index] == emp_id:
5.         return True
6.     return search_employee(emp_list, emp_id, index + 1)
7.
8. employees = [101, 105, 108, 112]
9. print(search_employee(employees, 108))
```

OUTPUT:

True

EXPLANATION:

Base Cases: If the index reaches the length of the list, there are no more employee IDs to check, so the function returns False. If the current ID matches emp_id, it returns True.

Recursive Step: When the current ID is not the required one, the function calls itself with the next index and continues the search.

ERROR TABLE:

SNO	ERROR	SOLUTION
No execution errors were encountered.		

d) In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number.

CODE:

```
1. def calculate_factorial(n):
2.     if n == 0 or n == 1:
3.         return 1
4.     else:
5.         return n * calculate_factorial(n - 1)
6.
7. print(calculate_factorial(5))
```

OUTPUT:

120

EXPLANATION:

Base Case: Factorial of 0 and 1 is defined as 1. Therefore, the function returns 1 when n reaches either of these values.

Recursive Step: For values greater than 1, the function multiplies n by the factorial of n - 1. The calls continue until the base case is reached.

ERROR TABLE:

SNO	ERROR	SOLUTION
No execution errors were encountered.		

e) Write a recursive python program to print the first n terms of fibonacci series.

CODE:

```
def fibonacci(n):
    if n <= 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)

def print_fibonacci_series(terms):
    for i in range(terms):
        print(fibonacci(i), end=" ")
    print()

print_fibonacci_series(7)
```

OUTPUT:

0 1 1 2 3 5 8

EXPLANATION:

Base Cases: The function directly gives 0 for the 0th Fibonacci term and 1 for the 1st term. These values stop further recursive calls.

Recursive Step: For every term after the first two, the function adds the previous two Fibonacci values by calling itself for n - 1 and n - 2.

ERROR TABLE:

SNO	ERROR	SOLUTION
No execution errors were encountered.		

LAB-2

a)Program to implement LINEAR SEARCH

CODE

```
1. def linear_search(arr, target):
2.     for i in range(len(arr)):
3.         if arr[i] == target:
4.             return i
5.     return -1
6.
7. my_array = [10, 23, 45, 70, 11, 15]
8. target_val = 70
9. print(linear_search(my_array, target_val))
```

OUTPUT:

3

EXPLANATION:

How it works: Linear search examines the elements one after another, beginning with the first element and continuing toward the end.

Condition: At each position, the current element is compared with the target value.

Result: If the target is found, its index is returned immediately. If every element is checked without a match, the function returns -1.

Time Complexity: O(n), because in the worst case the algorithm may need to inspect every element in the list.

ERROR TABLE:

SNO	ERROR	SOLUTION
No execution errors were encountered.		

b)Program to implement BINARY SEARCH

(1) With Sorted List:

CODE:

```
def binary_search_sorted(arr, target):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return -1

sorted_array = [11, 22, 34, 45, 55, 67, 89]
target_val = 45
print(binary_search_sorted(sorted_array, target_val))
```

OUTPUT:

3

EXPLANATION:

How it works: Binary search works on a sorted list by repeatedly reducing the portion of the list that needs to be examined.

Condition: The middle position of the current search range is calculated and its value is compared with the target.

- If the middle value equals the target, its index is returned.
- If the target is greater than the middle value, the search continues in the right half by setting low to mid + 1.
- If the target is smaller than the middle value, the search continues in the left half by setting high to mid - 1.

Result: The process repeats until the target is found or the search range becomes empty. In the latter case, -1 is returned.

Time Complexity: O(log n), since each iteration removes roughly half of the remaining search elements.

ERROR TABLE:

SNO	ERROR	SOLUTION

No execution errors were encountered.

(2) Without Sorted List:

CODE:

```
 1. def binary_search_unsorted(arr, target):
 2.     arr.sort()
 3.
 4.     low = 0
 5.     high = len(arr) - 1
 6.
 7.     while low <= high:
 8.         mid = (low + high) // 2
 9.         if arr[mid] == target:
10.             return mid
11.         elif arr[mid] < target:
12.             low = mid + 1
13.         else:
14.             high = mid - 1
15.
16.     return -1
17.
18. unsorted_array = [67, 11, 89, 22, 55, 45, 34]
19. target_val = 45
20. print(binary_search_unsorted(unsorted_array, target_val))
```

OUTPUT:

3

EXPLANATION:

How it works: Binary search needs sorted data, so the unsorted list must first be arranged before the search can be performed.

Condition: The statement arr.sort() changes the list into ascending order before the binary search begins.

Result: After sorting, the same binary-search procedure is used. The returned index belongs to the sorted list, so it may differ from the target's original index.

Time Complexity: O(n log n) overall because sorting takes O(n log n), while the binary search itself takes O(log n).

ERROR TABLE:

SNO	ERROR	SOLUTION
No execution errors were encountered.		

WEEK-3

a)Write a Program for BUBBLE SORT

CODE:

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr
```

OUTPUT:

[11, 12, 22, 25, 34, 64, 90]

EXPLANATION;

Bubble sort repeatedly compares neighboring elements and swaps them whenever the left value is greater than the right value. After each pass, the largest remaining value moves toward the end.

- Outer Loop: The outer loop controls the number of passes. After every pass, one more largest element reaches its correct final position.
- Inner Loop: The inner loop compares adjacent elements only within the unsorted portion. The range becomes smaller after each pass because the last elements are already sorted.
- Swap: When two neighboring values are in the wrong order, their positions are exchanged so that the smaller value moves left.
- Optimization: The swapped variable records whether any exchange occurred. If a complete pass makes no swaps, the list is already sorted and the algorithm stops early.

ERROR TABLE:

SNO	ERROR	SOLUTION
In **Bubble Sort** CODE I said to the computer to check every single position until the very end.		

b)Write a program for INSERTION SORT

CODE:

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and key < arr[j]:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr

my_array = [12, 11, 13, 5, 6]
print(insertion_sort(my_array))
```

OUTPUT:

[5, 6, 11, 12, 13]

EXPLANATION:

Insertion sort grows a sorted section of the array one element at a time. Each new element is placed into the correct position among the elements that are already sorted.

- Outer Loop: The process starts from index 1 because the first element alone can be treated as a sorted section. The current value is stored in key.
- While Loop: Elements larger than key are shifted one position to the right while moving backward through the sorted section.
- Insertion: When the correct position is reached, key is placed into the empty position created by the shifts.

ERROR TABLE:

SNO	ERROR	SOLUTION
made an error in updating the index while comparing the key with previous elements.		

c)Write a program for SELECTION SORT;

CODE:

```
def selection_sort(arr):
    for i in range(len(arr)):
        min_idx = i
        for j in range(i + 1, len(arr)):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr

my_array = [64, 25, 12, 22, 11]
print(selection_sort(my_array))
```

OUTPUT:

[11, 12, 22, 25, 64]

EXPLANATION:

Selection sort repeatedly finds the smallest value in the unsorted portion and places it at the beginning of that portion. This gradually expands the sorted section.

- Outer Loop: Index i marks the first position of the unsorted section. The code initially treats the element at i as the smallest candidate.
- Inner Loop: The remaining elements are checked to find whether a value smaller than the current minimum exists.
- Updating the Minimum: Whenever a smaller value is found, min_idx is changed to store its position.
- Swap: After the scan is complete, the smallest found element is exchanged with the element at index i, putting it into its final position.

ERROR TABLE:

SNO	ERROR	SOLUTION
made an error in the loop range, which caused the last element to be skipped.		

LAB – 4

4A..Write a Python program to implement **Merge Sort**, where the elements are taken from the user.

PROGRAM:

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
n = int(input("Enter number of elements: "))
arr = []
for i in range(n):
    arr.append(int(input("Enter element: ")))
print("Original array:", arr)
arr = merge_sort(arr)
print("Sorted array:", arr)
```

OUTPUT:

Enter number of elements: 5

Enter element: 10

Enter element: 2

Enter element: 11

Enter element: 8

Enter element: 16

Original array: [10, 2, 11, 8, 16]

Sorted array: [2, 8, 10, 11, 16]

EXPLANATION:

- First, the user enters the number of elements.
- The elements are stored in a list.
- merge_sort() checks the base condition len(arr) <= 1.
- The list is divided into two halves using mid.
- Both halves are recursively sorted.
- The two sorted halves are merged by comparing their elements.
- The final sorted list is displayed

ERRORS FACED:

- Mistakes in the main logic.
- Errors in conditions.

4B. Write a Python program to implement **Quick Sort**, where the elements are taken from the user

Aim

Write a Python program to implement **Quick Sort**, where the elements are taken from the user

PROGRAM:

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[0]
    left = []
    right = []
    for x in arr[1:]:
        if x <= pivot:
            left.append(x)
        else:
            right.append(x)
    return quick_sort(left) + [pivot] + quick_sort(right)
n = int(input("Enter number of elements: "))
arr = []
for i in range(n):
    arr.append(int(input("Enter element: ")))
print("Original array:", arr)
arr = quick_sort(arr)
print("Sorted array:", arr)
```

OUTPUT:

Enter number of elements: 6

Enter element: 45

Enter element: 12

Enter element: 78

Enter element: 3

Enter element: 25

Enter element: 60

Original array: [45, 12, 78, 3, 25, 60]

Sorted array: [3, 12, 25, 45, 60, 78]

EXPLANATION:

- The program takes the elements from the user.
- quick_sort() checks whether the list has zero or one element.
- The first element is selected as the **pivot**.
- Elements smaller than or equal to the pivot are placed in left.
- Elements greater than the pivot are placed in right.
- Quick Sort is recursively applied to both lists.
- Finally, left + pivot + right produces the sorted list.

ERRORS FACED:

- Incorrect selection of pivot.
- Error while dividing elements into left and right lists.

WEEK – 5

5A. Write a program to implement **Singly Linked List** with the following operations:

- Create linked list
- Insert at beginning
- Insert at end
- Insert at index
- Delete by value
- Delete first node
- Delete last node
- Count number of nodes
- Display / traverse
- Exit

Aim

To write a Python program to implement a Singly Linked List with insertion, deletion, counting and traversal operations.

PROGRAM:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class SinglyLinkedList:
    def __init__(self):
        self.head = None
    def insert_beginning(self, data):
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node
```

```
    def insert_end(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        temp = self.head
        while temp.next:
            temp = temp.next
        temp.next = new_node
    def insert_index(self, data, index):
        if index == 0:
            self.insert_beginning(data)
            return
        temp = self.head
        for _ in range(index - 1):
            if temp is None:
                print("Invalid index")
                return
            temp = temp.next
        if temp is None:
            print("Invalid index")
            return
        new_node = Node(data)
        new_node.next = temp.next
        temp.next = new_node
    def delete_value(self, value):
        if self.head is None:
            return
        if self.head.data == value:
```

```
            self.head = self.head.next
            return
        temp = self.head
        while temp.next and temp.next.data != value:
            temp = temp.next
        if temp.next:
            temp.next = temp.next.next
    def delete_first(self):
        if self.head:
            self.head = self.head.next
    def delete_last(self):
        if self.head is None:
            return
        if self.head.next is None:
            self.head = None
            return
        temp = self.head
        while temp.next.next:
            temp = temp.next
        temp.next = None
    def count(self):
        count = 0
        temp = self.head
        while temp:
            count += 1
            temp = temp.next
        return count
    def display(self):
        temp = self.head
```

```
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")

sll = SinglyLinkedList()

sll.insert_end(10)
sll.insert_end(20)
sll.insert_end(30)
sll.insert_end(40)

print("Linked List:")
sll.display()

sll.insert_beginning(5)
print("After inserting at beginning:")
sll.display()

sll.insert_index(25, 3)
print("After inserting at index:")
sll.display()

sll.delete_value(25)
print("After deleting value 25:")
sll.display()

print("Number of nodes:", sll.count())
```

OUTPUT:

The program successfully creates a **Singly Linked List** and performs the required operations such as insertion at the beginning, insertion at the end, insertion at a specified index, deletion by value, deletion of the first and last nodes, counting the number of nodes, and displaying the linked list.

For example, the linked list is displayed in the form:

10 → 20 → 30 → 40 → None

The number of nodes is also displayed after performing the required operations.

EXPLANATION:

- Each node contains **data** and a **next** reference.
- head stores the reference to the first node.
- insert_beginning() adds a node before the current first node.
- insert_end() traverses to the last node and adds a new node.
- insert_index() inserts a node at a specified position.
- delete_value() searches for a value and removes that node.
- delete_first() moves head to the next node.
- delete_last() removes the final node.
- count() traverses the list and counts the nodes.
- display() traverses from head and prints each node.

ERRORS FACED:

Errors in node linking.

5B. Write a program to implement **Doubly Linked List** with the following operations:

- Create linked list
- Insert at beginning
- Insert at end
- Insert at index
- Delete by value
- Delete first node
- Delete last node
- Count number of nodes
- Display / traverse
- Exit

AIM:

To write a Python program to implement a Doubly Linked List with insertion, deletion, counting and traversal operations.

PROGRAM:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None
class DoublyLinkedList:
    def __init__(self):
        self.head = None
    def insert_beginning(self, data):
        new_node = Node(data)
        new_node.next = self.head
        if self.head:
```

```
            self.head.prev = new_node
        self.head = new_node
    def insert_end(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        temp = self.head
        while temp.next:
            temp = temp.next
        temp.next = new_node
        new_node.prev = temp
    def insert_index(self, data, index):
        if index == 0:
            self.insert_beginning(data)
            return
        temp = self.head
        for _ in range(index - 1):
            if temp is None:
                print("Invalid index")
                return
            temp = temp.next
        if temp is None:
            print("Invalid index")
            return
        new_node = Node(data)
        new_node.next = temp.next
        new_node.prev = temp
        if temp.next:
```

```
            temp.next.prev = new_node
        temp.next = new_node
    def delete_value(self, value):
        temp = self.head
        while temp and temp.data != value:
            temp = temp.next
        if temp is None:
            return
        if temp.prev:
            temp.prev.next = temp.next
        else:
            self.head = temp.next
        if temp.next:
            temp.next.prev = temp.prev
    def delete_first(self):
        if self.head is None:
            return
        self.head = self.head.next
        if self.head:
            self.head.prev = None
    def delete_last(self):
        if self.head is None:
            return
        temp = self.head
        while temp.next:
            temp = temp.next
        if temp.prev:
            temp.prev.next = None
        else:
```

```
            self.head = None
    def count(self):
        count = 0
        temp = self.head
        while temp:
            count += 1
            temp = temp.next
        return count
    def display(self):
        temp = self.head
        while temp:
            print(temp.data, end=" <-> ")
            temp = temp.next
        print("None")
dll = DoublyLinkedList()
dll.insert_end(10)
dll.insert_end(20)
dll.insert_end(30)
dll.insert_end(40)
print("Doubly Linked List:")
dll.display()
dll.insert_beginning(5)
print("After inserting at beginning:")
dll.display()

dll.insert_index(25, 3)
print("After inserting at index:")
dll.display()
```

```
dll.delete_value(25)

print("After deleting value 25:")

dll.display()

print("Number of nodes:", dll.count())
```

OUTPUT:

The program successfully creates a **Doubly Linked List** and performs the required insertion, deletion, counting, and traversal operations. The elements are displayed with links between the previous and next nodes.

For example, the linked list is displayed in the form:

10 ↔ 20 ↔ 30 ↔ 40 ↔ None

EXPLANATION:

- A DLL node contains **three parts: prev, data, and next**.
- prev points to the previous node.
- next points to the next node.
- head points to the first node.
- The list can therefore be traversed in both forward and backward directions.
- During insertion, both prev and next links must be updated.
- During deletion, the neighbouring nodes' links must be adjusted.
- count() traverses the list and counts the nodes.
- display() traverses from head and prints the elements using <->.

ERRORS FACED:

Errors in previous and next linkin

WEEK – 6

6A. Write a Python program to implement stack operations such as **push, pop, peek, and display using an array and a linked list**.

AIM:

Write a Python program to implement stack operations such as **push, pop, peek, and display using an array and a linked list**.g.

PROGRAM:

```
class Stack:
    def __init__(self, size):
        self.size = size
        self.stack = [None] * size
        self.top = -1

    def push(self, item):
        if self.top == self.size - 1:
            print("Stack Overflow")
        else:
            self.top += 1
            self.stack[self.top] = item

    def pop(self):
        if self.top == -1:
            print("Stack Underflow")
        else:
            item = self.stack[self.top]
            self.top -= 1
            print(item, "popped from stack")
```

```
    def peek(self):
        if self.top == -1:
            print("Stack Underflow")
        else:
            print("Top element:", self.stack[self.top])

    def display(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            for i in range(self.top, -1, -1):
                print(self.stack[i], end=" ")
            print()
s = Stack(5)

s.push(10)
s.push(20)
s.push(30)

s.display()
s.peek()
s.pop()
s.display()
```

EXPLANATION:

A stack follows the **LIFO (Last In, First Out)** principle. In an array-based implementation, the stack is stored using an array/list and a **top** position is used to identify the top element.

- **Push** – adds an element to the top of the stack.

- **Pop** – removes the top element.
- **Peek** – displays the top element without removing it.
- **Display** – displays the elements of the stack.
- **Overflow** occurs when a fixed-size array stack is full.
- **Underflow** occurs when a pop operation is performed on an empty stack.

OUTPUT:

The program successfully implements stack operations using an array. The elements **10, 20 and 30** are pushed into the stack and displayed from top to bottom as **30 20 10**. The peek operation displays **30** as the top element. When the pop operation is performed, **30** is removed from the stack. The remaining stack elements are displayed as **20 10**.

EXPLANATION

A stack follows the **LIFO (Last In, First Out)** principle. In an array-based implementation, the stack is stored using an array/list and a **top** position is used to identify the top element.

- **Push** – adds an element to the top of the stack.
- **Pop** – removes the top element.
- **Peek** – displays the top element without removing it.
- **Display** – displays the elements of the stack.
- **Overflow** occurs when a fixed-size array stack is full.
- **Underflow** occurs when a pop operation is performed on an empty stack.

ERRORS FACED

1. Mistakes in main logic
2. Errors in condition

6B. To implement stack operations such as **push, pop, peek, and display using a linked list**.

AIM:

To implement stack operations such as **push, pop, peek, and display using a linked list**.

PROGRAM:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None



class Stack:
    def __init__(self):
        self.top = None

    def push(self, data):
        new = Node(data)
        new.next = self.top
        self.top = new

    def pop(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            temp = self.top
            print(temp.data, "popped from stack")
            self.top = self.top.next
```

```
    def peek(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            print("Top element:", self.top.data)

    def display(self):
        temp = self.top
        while temp is not None:
            print(temp.data, end=" ")
            temp = temp.next
        print()


s = Stack()

s.push(10)
s.push(20)
s.push(30)

s.display()
s.peek()
s.pop()
s.display()
```

OUTPUT:

The program successfully implements stack operations using a linked list. The elements **10, 20 and 30** are pushed into the stack and displayed from top to bottom as **30 20 10**. The peek operation displays **30** as the top element. When the pop operation is performed, **30** is removed from the stack. The remaining stack elements are displayed as **20 10**.

EXPLANATION:

A stack can also be implemented using a linked list. Each node contains **data** and a **next** reference. The **TOP pointer** points to the topmost node.

- **Push:** A new node is created and added at the top.
- **Pop:** The top node is removed and TOP is moved to the next node.
- **Peek:** The data of the top node is accessed without removing it.
- **Display:** The stack elements are traversed starting from TOP.
- When the stack is empty, **TOP = None** and pop results in **Stack Underflow**.

ERRORS FACED:

1. Errors in push and pop operations
2. Errors in top pointer handling

WEEK – 7

7a. Queue implementation using Linked List.

AIM:

Queue implementation using Linked List.

Program:

```
class Queue:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1
    def enqueue(self, x):
        if self.rare== self.size - 1:
            print("Queue Overflow")
            return
        if self.front == -1:
            self.front = 0
        self.rear += 1
        self.queue[self.rear] = x
    def dequeue(self):
        if self.front == -1 or self.front > self.rear:
            print("Queue is Empty")
            return
        x = self.queue[self.front]
        self.front += 1
        print("Deleted element:", x)
        if self.front > self.rear:
            self.front = -1
```

```
            self.rear = -1
    def peek(self):
        if self.front == -1:
            print("Queue is Empty")
        else:
            print("Front element:", self.queue[self.front])
    def display(self):
        if self.front == -1:
            print("Queue is Empty")
            return
        print("Queue:", end=" ")
        for i in range(self.front, self.rear + 1):
            print(self.queue[i], end=" ")
        print()
size = int(input("enter the size of the queue: "))
q = Queue(size)
print("queue using arrays")
while True:
    print("1.enqueue")
    print("2.dequeue")
    print("3.peek")
    print("4.display")
    print("5.exit")
    selection = int(input("select the index to perform the operation : "))
    if selection == 1:
        x = int(input("enter the element to enqueue: "))
        q.enqueue(x)
    elif selection == 2:
        q.dequeue()
```

```
    elif selection == 3:
        q.peek()
    elif selection == 4:
        q.display()
    elif selection == 5:
        break
    else:
        print("invalid selection")
```

Output:

```
enter the size of the queue:5
 queue using arrays
1.enqueue
 2.dequeue
3.peek
 4.display
 5.exit
select the index to perform the operation : 1
 enter the element to enqueue: 10
select the index to perform the operation :
```

Explanation:

- A queue follows **FIFO (First In, First Out)**.
- **Enqueue:** inserts an element at the rear.
- **Dequeue:** removes an element from the front.
- **Peek:** displays the front element without deleting it.
- **Display:** prints all elements from front to rear.
- Queue overflow occurs when rear == size - 1.

Errors faced:

- Forgetting front = 0 on first insertion
- Forgetting rear += 1

7b. Queue Using Linked List

Aim:

Queue Using Linked List

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class Queue:
    def __init__(self):
        self.front = None
        self.rear = None
    def enqueue(self, x):
        new = Node(x)
        if self.rear is None:
            self.front = self.rear = new
        else:
            self.rear.next = new
            self.rear = new
    def dequeue(self):
        if self.front is None:
            print("Queue is Empty")
            return
        x = self.front.data
        self.front = self.front.next
        print("Deleted element:", x)
        if self.front is None:
            self.rear = None
    def peek(self):
```

```
        if self.front is None:
            print("Queue is Empty")
        else:
            print("Front element:", self.front.data)
    def display(self):
        if self.front is None:
            print("Queue is Empty")
            return
        temp = self.front
        print("Queue:", end=" ")
        while temp is not None:
            print(temp.data, end=" ")
            temp = temp.next
        print()
q = Queue()
print("queue using linked list")
while True:
    print("1.enqueue")
    print("2.dequeue")
    print("3.peek")
    print("4.display")
    print("5.exit")
    selection = int(input("select the index to perform the operation : "))
    if selection == 1:
        x = int(input("enter the element to enqueue: "))
        q.enqueue(x)
    elif selection == 2:
        q.dequeue()
    elif selection == 3:
```

```
        q.peek()
    elif selection == 4:
        q.display()
    elif selection == 5:
        break
    else:
        print("invalid selection")
```

Output:

```
queue using linked list
1.enqueue
2.dequeue
3.peek
4.display
5.exit
select the index to perform the operation : 1
enter the element to enqueue: 10

select the index to perform the operation : 1
enter the element to enqueue: 20

select the index to perform the operation : 4
Queue: 10 20

select the index to perform the operation : 3
Front element: 10

select the index to perform the operation : 2
Deleted element: 10
```

select the index to perform the operation : 4

Queue: 20

Explanation:

- This Python program implements a Queue using a Linked List based on the FIFO (First In, First Out) rule.
- The Node class creates individual nodes that hold the data and a link (next) to the next element.
- The Queue class manages the queue using front to track the first element and rear to track the last element.
- The enqueue(x) function adds a new element to the end of the queue.
- The dequeue() function removes and displays the first element from the front.
- The peek() function displays the front element without removing it.
- The display() function prints all elements in the queue from front to back.
- An interactive menu loop allows the user to perform these operations until they choose to exit.

Errors faced:

- Writing rear = new before linking
- Writing rear.next = rear

7c. program for performing the basic operations using circular queue

Aim:

program for performing the basic operations using circular queue

program:

```
class Queue:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1
    def enqueue(self, x):
        if (self.rear + 1) % self.size == self.front:
            print("Queue Overflow")
            return
        if self.front == -1:
            self.front = 0
            self.rear = 0
        else:
            self.rear = (self.rear + 1) % self.size
        self.queue[self.rear] = x

    def dequeue(self):
        if self.front == -1:
            print("Queue is Empty")
            return

        x = self.queue[self.front]
        print("Deleted element:", x)
        if self.front == self.rear:
```

```
            self.front = -1
            self.rear = -1
        else:
            self.front = (self.front + 1) % self.size
    def peek(self):
        if self.front == -1:
            print("Queue is Empty")
        else:
            print("Front element:", self.queue[self.front])
    def display(self):
        if self.front == -1:
            print("Queue is Empty")
            return
        print("Queue:", end=" ")
        i = self.front
        while True:
            print(self.queue[i], end=" ")
            if i == self.rear:
                break
            i = (i + 1) % self.size
        print()
size = int(input("enter the size of the queue: "))
q = Queue(size)
print("circular queue")
while True:
    print("1.enqueue")
    print("2.dequeue")
    print("3.peek")
    print("4.display")
```

```
    print("5.exit")
    selection = int(input("select the index to perform the operation : "))
    if selection == 1:
        x = int(input("enter the element to enqueue: "))
        q.enqueue(x)
    elif selection == 2:
        q.dequeue()
    elif selection == 3:
        q.peek()
    elif selection == 4:
        q.display()
    elif selection == 5:
        break
    else:
        print("invalid selection")
```

Output:

```
enter the size of the queue: 3
circular queue
1.enqueue
2.dequeue
3.peek
4.display
5.exit
select the index to perform the operation : 1
enter the element to enqueue: 10
1.enqueue
2.dequeue
```

3.peek

4.display

5.exit

select the index to perform the operation :

Explanation:

- This Python program implements a Circular Queue using a fixed-size array (list).
- It follows the FIFO (First In, First Out) principle and reuses empty spaces by connecting the end of the array back to the beginning.
- The Queue class uses front and rear index pointers initialized to -1, along with the modulo operator (%) for circular movement.
- The enqueue(x) function checks for queue overflow and adds a new element to the rear position.
- The dequeue() function checks for queue underflow, removes the element at front, and resets pointers when the queue becomes empty.
- The peek() function displays the element at the front position without removing it.
- The display() function traverses circularly from front to rear and prints all current elements.
- An interactive menu loop prompts the user to enter a fixed size and select operations dynamically until they choose to exit.

Errors faced:

- Using linear queue full condition
- Forgetting first insertion case

WEEK – 8

8a. write a python program to implement a binary tree with its operations using a array and a linked list

AIM: to write a python program to implement a binary tree with its operations using a array and a linked list

Arrays:

Code:

```
class BinaryTree:
    def __init__(self, size):
        self.tree = [None] * size
        self.size = size

    # Insert a node
    def insert(self, value):
        for i in range(self.size):
            if self.tree[i] is None:
                self.tree[i] = value
                return
        print("Tree is full")

    # Search a node
    def search(self, value):
        for i in range(self.size):
            if self.tree[i] == value:
                return i
        return -1

    # Inorder Traversal
    def inorder(self, i=0):
        if i >= self.size or self.tree[i] is None:
```

```
            return

        self.inorder(2 * i + 1)
        print(self.tree[i], end=" ")
        self.inorder(2 * i + 2)

    # Preorder Traversal
    def preorder(self, i=0):
        if i >= self.size or self.tree[i] is None:
            return

        print(self.tree[i], end=" ")
        self.preorder(2 * i + 1)
        self.preorder(2 * i + 2)

    # Postorder Traversal
    def postorder(self, i=0):
        if i >= self.size or self.tree[i] is None:
            return

        self.postorder(2 * i + 1)
        self.postorder(2 * i + 2)
        print(self.tree[i], end=" ")

    # Display array
    def display(self):
        print(self.tree)
```

```
# Main program
bt = BinaryTree(10)

bt.insert(25)
bt.insert(15)
bt.insert(40)
bt.insert(10)
bt.insert(20)
bt.insert(30)
bt.insert(50)

print("Array representation:")
bt.display()

print("\nInorder:")
bt.inorder()

print("\nPreorder:")
bt.preorder()

print("\nPostorder:")
bt.postorder()

print("\n\nSearching for 30:")

position = bt.search(30)

if position != -1:
    print("30 found at index", position)
```

```
else:
    print("30 not found")
```

Output:

Array representation:

[25, 15, 40, 10, 20, 30, 50, None, None, None]

Inorder:

10 15 20 25 30 40 50

Preorder:

25 15 10 20 40 30 50

Postorder:

10 20 15 30 50 40 25

Searching for 30:

30 found at index 5

8B. Using Linked List:

AIM: binary trees using linked list

program

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None



class BinaryTree:
    def __init__(self):
        self.root = None

    # Insert a node
    def insert(self, value):
        new_node = Node(value)

        if self.root is None:
            self.root = new_node
            return

        queue = [self.root]

        while queue:
            current = queue.pop(0)

            if current.left is None:
```

```
            current.left = new_node
            return
        else:
            queue.append(current.left)

        if current.right is None:
            current.right = new_node
            return
        else:
            queue.append(current.right)

    # Search a node
    def search(self, value):
        if self.root is None:
            return False

        queue = [self.root]

        while queue:
            current = queue.pop(0)

            if current.data == value:
                return True

            if current.left:
                queue.append(current.left)

            if current.right:
                queue.append(current.right)
```

```
        return False

    # Inorder Traversal
    def inorder(self, node):
        if node is not None:
            self.inorder(node.left)
            print(node.data, end=" ")
            self.inorder(node.right)

    # Preorder Traversal
    def preorder(self, node):
        if node is not None:
            print(node.data, end=" ")
            self.preorder(node.left)
            self.preorder(node.right)

    # Postorder Traversal
    def postorder(self, node):
        if node is not None:
            self.postorder(node.left)
            self.postorder(node.right)
            print(node.data, end=" ")

    # Level Order Traversal
    def level_order(self):
        if self.root is None:
            return
```

```
        queue = [self.root]

        while queue:
            current = queue.pop(0)

            print(current.data, end=" ")

            if current.left:
                queue.append(current.left)

            if current.right:
                queue.append(current.right)


# Main program
bt = BinaryTree()

bt.insert(25)
bt.insert(15)
bt.insert(40)
bt.insert(10)
bt.insert(20)
bt.insert(30)
bt.insert(50)

print("Inorder:")
bt.inorder(bt.root)

print("\nPreorder:")
```

```
bt.preorder(bt.root)

print("\nPostorder:")
bt.postorder(bt.root)

print("\nLevel Order:")
bt.level_order()

print("\n\nSearching for 30:")

if bt.search(30):
    print("30 found")
else:
    print("30 not found")
```

Output:

```
Inorder:
10 15 20 25 30 40 50
Preorder:
25 15 10 20 40 30 50
Postorder:
10 20 15 30 50 40 25
Level Order:
25 15 40 10 20 30 50

Searching for 30:
30 found
```

Explanation:

A **Binary Tree** is a non-linear data structure in which each node can have a maximum of two children, called the **left child** and **right child**. It can be implemented using an **array** or a **linked list**. The main operations include **insertion, searching, and traversal**. The tree can be traversed using **Inorder, Preorder, Postorder, and Level Order** methods. In array representation, nodes are stored using index positions, whereas in linked-list representation, each node contains **data and pointers to its left and right children**. Both methods are useful for storing and processing hierarchical data.

Inorder: Left → Root → Right

Preorder: Root → Left → Right

Postorder: Left → Right → Root

Level Order: Visit nodes level by level

Error Table

S.no	Error	Explanation
1	Using 2*i+1 incorrectly	Left = 2*i+1, Right = 2*i+2
2	Using wrong variable name	Use the same variable name throughout the program.

WEEK – 9

Q) Write a Python program to implement a Binary Search Tree with its operations and traversals.

AIM:

To write a Python program to implement a Binary Search Tree with its operations and traversals.

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None



class BinarySearchTree:
    # Insert a node
    def insert(self, root, data):
        if root is None:
            return Node(data)
        if data < root.data:
            root.left = self.insert(root.left, data)
        elif data > root.data:
            root.right = self.insert(root.right, data)
        return root

    # Search for a node
    def search(self, root, data):
        if root is None:
            return False
        if root.data == data:
            return True
        elif data < root.data:
            return self.search(root.left, data)
        else:
            return self.search(root.right, data)

    # Find minimum value node
    def find_min(self, root):
        current = root
        while current.left is not None:
            current = current.left
        return current

    # Delete a node
    def delete(self, root, data):
        if root is None:
            return root
        if data < root.data:
            root.left = self.delete(root.left, data)
```

```
        elif data > root.data:
            root.right = self.delete(root.right, data)
        else:
            # Case 1: No child
            if root.left is None and root.right is None:
                return None
            # Case 2: Only right child
            elif root.left is None:
                return root.right
            # Case 3: Only left child
            elif root.right is None:
                return root.left
            # Case 4: Two children
            else:
                temp = self.find_min(root.right)
                root.data = temp.data
                root.right = self.delete(root.right, temp.data)
        return root

    # Inorder: Left -> Root -> Right
    def inorder(self, root):
        if root is not None:
            self.inorder(root.left)
            print(root.data, end=" ")
            self.inorder(root.right)

    # Preorder: Root -> Left -> Right
    def preorder(self, root):
        if root is not None:
            print(root.data, end=" ")
            self.preorder(root.left)
            self.preorder(root.right)

    # Postorder: Left -> Right -> Root
    def postorder(self, root):
        if root is not None:
            self.postorder(root.left)
            self.postorder(root.right)
            print(root.data, end=" ")



# Main program
bst = BinarySearchTree()
root = None

while True:
    print("\n\n--- BINARY SEARCH TREE ---")
    print("1. Insert")
    print("2. Search")
    print("3. Delete")
    print("4. Inorder Traversal")
    print("5. Preorder Traversal")
    print("6. Postorder Traversal")
    print("7. Exit")
```

```
choice = int(input("Enter your choice: "))

if choice == 1:
    data = int(input("Enter value to insert: "))
    root = bst.insert(root, data)
    print("Node inserted successfully.")
elif choice == 2:
    data = int(input("Enter value to search: "))
    if bst.search(root, data):
        print("Element found.")
    else:
        print("Element not found.")
elif choice == 3:
    data = int(input("Enter value to delete: "))
    root = bst.delete(root, data)
    print("Node deleted successfully.")
elif choice == 4:
    print("Inorder Traversal:")
    bst.inorder(root)
elif choice == 5:
    print("Preorder Traversal:")
    bst.preorder(root)
elif choice == 6:
    print("Postorder Traversal:")
    bst.postorder(root)
elif choice == 7:
    print("Program terminated.")
    break
else:
    print("Invalid choice.")
```

Output:

The program displays a menu to insert, search, delete, and perform inorder, preorder, and postorder traversals. The actual output depends on the values entered by the user.

Explanation:

This Python program implements a Binary Search Tree (BST) using nodes and supports insertion, searching, and deletion of elements. It also performs inorder, preorder, and postorder traversals. In a BST, smaller values are stored in the left subtree and larger values in the right subtree. The program uses a menu-driven approach to perform different operations.

Error Table:

S.no	Error	Explanation
1	Using wrong comparison operator	Use < for the left subtree and > for the right subtree.
2	Forgetting return root	Return the updated root after insertion or deletion.

3	Using wrong traversal order	Inorder = Left → Root → Right; Preorder = Root → Left → Right; Postorder = Left → Right → Root.